

Execution-Aware A* Search for Cross-Exchange Stablecoin Arbitrage

Anonymous Author(s)
Paper ID *****

Abstract

Cross-exchange cryptocurrency arbitrage enables low-risk profit from price discrepancies across exchanges, yet existing approaches predominantly employ negative cycle detection algorithms (e.g., Bellman–Ford) that target opportunity identification rather than execution feasibility. We introduce an execution-aware pathfinding framework for stablecoin cross-exchange arbitrage using A* search with domain-specific guidance heuristics. The problem is modelled as a weighted directed graph where nodes represent (exchange, stablecoin) pairs across 12 major centralized exchanges (41 nodes, 864 edges) and edges encode trades and transfers with real-world execution costs including trading fees, order-book slippage, withdrawal fees, gas costs, transfer delays, and exchange reliability risks. We propose three guidance heuristics, h_1 (liquidity), h_2 (slippage), h_4 (chain congestion and exchange risk via Weighted A*), and a multi-start search-control strategy. Evaluation over 7,200 search instances in an 8-hour overnight campaign, graph scaling from 4 to 12 exchanges, sensitivity sweeps, and cached-graph benchmarks reveals differentiated heuristic roles: h_2 reduces node expansions by 29% relative to Dijkstra while matching profit, and h_4 provides risk-aware guidance at Dijkstra-class latency (P99 = 33 ms) with zero API overhead. The execution-aware graph formulation itself is the primary contribution: Dijkstra alone achieves 100% success across overnight runs, demonstrating that embedding real-world frictions into edge weights makes any shortest-path algorithm a viable arbitrage planner. Code and data are available at <https://anonymous.4open.science/r/Stablecoin-CrossExchange-Arbitrage-6CB2>.

Keywords: stablecoin arbitrage, A* search, cross-exchange trading, execution-aware pathfinding, cryptocurrency, graph algorithms

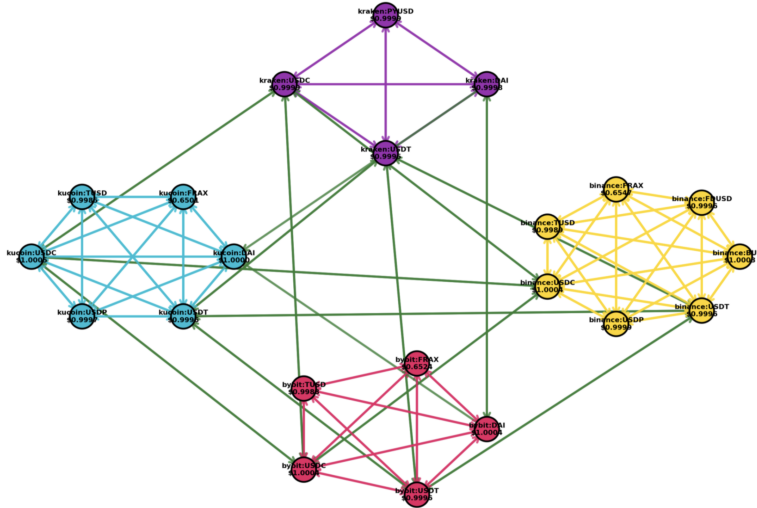


Figure 1. Overview of the cross-exchange arbitrage graph used in our experiments. Nodes represent (exchange, asset) pairs. Intra-exchange edges denote trading pairs, while inter-exchange edges represent asset transfers.

(emails omitted for review)

1. Introduction

1.1. Problem Statement and Research Questions

Cross-exchange stablecoin arbitrage exploits price differences for the same asset across centralized exchanges. Profitable opportunities depend not only on visible price spreads but also on fees, slippage, transfer delays, liquidity constraints, exchange reliability, and short arbitrage windows. We formulate this as a graph-planning problem and develop an automated route planner that computes complete, executable trade-and-transfer paths rather than simple pairwise price gaps. By incorporating trading fees, withdrawal costs, transfer latencies, liquidity depth, and live market data via CCXT [1], the system generates execution-ready arbitrage paths. Unlike prior detection tools [2], our approach integrates real-world constraints and applies A* search with domain-specific heuristics.

Our methodology addresses four research questions: (1) Do certain heuristics consistently outperform others on specific trading-instance classes? (2) How does each heuristic influence A*'s node-expansion behaviour? (3) How does performance change as structural difficulty increases (liquidity sparsity, high withdrawal fees, fragmented listings)? (4) When heuristics return suboptimal routes, how close are they to the best-known solution?

1.2. Why Cross-Exchange Stablecoin Arbitrage Matters

Stablecoins serve as the primary liquidity bridges in cryptocurrency markets. Because centralized exchanges operate independently, price discrepancies for the same USD-pegged asset frequently arise due to fragmented liquidity, withdrawal delays, blockchain congestion, and operational frictions. Unlike volatile cryptocurrencies, stablecoins operate within a bounded price band anchored to fiat currency, reframing arbitrage from speculative price prediction into a constrained execution-optimization problem where path-dependent execution costs dominate expected return.

During periods of macroeconomic instability, stablecoin flows intensify as capital migrates toward dollar-pegged assets, amplifying cross-exchange imbalances and increasing both spread frequency and execution complexity.

1.3. Key Challenges

Cross-exchange arbitrage is constrained by several real-world frictions. **Liquidity:** order-book depth determines deployable capital without market impact. **Slippage:** large orders incur nonlinear price impact, so theoretical profit often differs from realized profit. **Latency:** blockchain confirmations may exceed the arbitrage window. **Reliability:** exchanges may experience withdrawal suspensions or API instability. These constraints interact: faster chains may reduce latency but carry higher reliability risk. Our heuristic-guided A* framework models these trade-offs to prioritize routes that are not only theoretically profitable but realistically executable.

2. Related Work

Cryptocurrency arbitrage has been studied from multiple perspectives. Most prior work focuses on identifying arbitrage opportunities, while comparatively less attention has been devoted to modeling execution feasibility under real-world constraints.

Graph-theoretic arbitrage detection. The dominant formulation models financial markets as directed weighted graphs where the log transform $w(e) = -\log(r(e))$ converts multiplicative rates into additive costs, so arbitrage opportunities correspond to negative-weight cycles detectable by Bellman–Ford [3, 4]. Oantă and Coroiu [2] adopt this paradigm in *Crypto*

Advisor, aggregating real-time exchange data to detect arbitrage cycles. Similar graph-based formulations appear in DeFi systems [5], improved negative-cycle algorithms [6, 7], and high-frequency frameworks [8]. Across these works, the core objective is opportunity detection rather than execution-aware feasibility.

Execution feasibility and market frictions. Single-exchange optimal execution has been studied using reinforcement learning and stochastic control [9, 10]. Cross-exchange arbitrage introduces additional constraints: blockchain transfer latency, withdrawal fees, liquidity fragmentation, and exchange reliability. Stablecoin markets are particularly sensitive to these frictions due to narrow spreads. Obadia et al. [11] show that on-chain arbitrage profits are highly sensitive to transaction ordering and confirmation latency; in the CEX-to-CEX setting, withdrawal processing and blockchain finality delays add further uncertainty.

Risk-aware pathfinding and slippage. Nikolova and Karger [12] study risk-averse pathfinding under stochastic edge weights, and Lim et al. [13] propose CVaR pathfinding to minimize worst-case tail cost. Our h_4 heuristic addresses a similar concern by penalizing unreliable exchanges and congested chains. For slippage, execution cost increases with order size in both AMMs and CEX order books. Our h_2 heuristic estimates slippage via live VWAP computation, avoiding parametric assumptions; more advanced models such as Almgren and Chriss [14] are left for future work.

Positioning. In contrast to prior work, our approach integrates feasibility modeling directly into A* search with domain-specific guidance heuristics, bridging opportunity detection and executable route construction for the low-margin stablecoin context.

3. Problem Formulation

3.1. Graph Representation

We formulate the stablecoin cross-exchange arbitrage problem as a directed weighted graph $G = (V, E)$. Each node $v \in V$ represents a unique stablecoin–exchange pair, written as $v = (\text{exchange}, \text{coin})$. Each directed edge $e \in E$ corresponds either to a trade executed on an exchange or a transfer of funds between exchanges.

Every edge has an associated effective exchange rate that incorporates all relevant costs (trading fees, withdrawal fees, spreads, and slippage). We assign each edge a weight $w(e) = -\log(\text{effective_rate}(e))$, converting multiplicative returns into additive costs and enabling shortest-path algorithms to evaluate profitability.

3.2. Path Profitability

Given a path $P = (v_0, v_1, \dots, v_k)$ with initial capital C_0 , the final capital is $C_k = C_0 \cdot \prod_{e \in P} r(e)$ where $r(e)$ denotes the effective rate on edge e . A path is profitable when $C_k - C_0 \geq \text{min_profit}$. In additive log space this becomes:

$$\sum_{e \in P} -\log(r(e)) \leq -\log\left(1 + \frac{\text{min_profit}}{C_0}\right).$$

The search problem is to find a path from start node v_0 to some node v_k such that $\prod_{e \in P} r(e) > 1$.

Goal Condition. We define the goal set $\mathcal{G} = \{v_k \in V \mid C_k > C_0\}$. Critically, we do *not* require $v_k = v_0$. This is an **open-path** formulation: the trader may end on any exchange–coin pair where the USD value exceeds the initial capital. This is justified because all assets are stablecoins pegged to USD (we enforce a $\pm 2\%$ price tolerance when building the graph), so profit at any terminal node is realized value. Standard arbitrage literature typically requires closed cycles, but this restriction is unnecessarily limiting in the stablecoin domain:

a profitable open path (e.g., starting with USDT on Kraken and ending with TUSD on KuCoin) yields the same economic outcome.

3.3. A* Search Formulation

We apply A* search with cost-to-reach $g(n) = \sum_{e \in \text{path}} -\log(r(e))$ and evaluation function $f(n) = g(n) + h(n)$, where $h(n)$ estimates remaining execution risk. The algorithm terminates when $C_n > C_0$.

Heuristic Design Philosophy. In our domain, the true remaining cost $h^*(n)$ is not well-defined because (i) the goal set depends on accumulated capital along the path, and (ii) market conditions change over time. We therefore design heuristics h_1 – h_4 as **guidance penalties** rather than admissible lower bounds. Each $h_i(n) \geq 0$ penalizes execution-risk features (illiquidity, slippage, chain congestion, exchange unreliability), steering A* toward paths more likely to remain profitable under real-world conditions.

For h_4 , we employ **Weighted A*** [15] with $f(n) = g(n) + w(n) \cdot h(n)$, where $w(n) = w_{\min} + (w_{\max} - w_{\min}) \cdot \rho(n)$ and $\rho(n) \in [0, 1]$ is a chain kickback risk score. When $w(n) = 1$ for all nodes, this reduces to standard A*.

Completeness. The algorithm is not complete: market data does not guarantee a profitable path from every starting node. Search terminates when a profitable path is found, the frontier is exhausted, or depth/time limits are reached.

3.4. Demonstrative Example

Consider a simplified arbitrage path with three nodes: kraken:USDT, kucoin:USDT, and kucoin:TUSD. Suppose the following edges with effective rates:

- kraken:USDT $\rightarrow$ kucoin:USDT: rate $r_1 = 0.9995$ (cross-exchange transfer)
- kucoin:USDT $\rightarrow$ kucoin:TUSD: rate $r_2 = 1.0042$ (intra-exchange trade)

The path kraken:USDT $\rightarrow$ kucoin:USDT $\rightarrow$ kucoin:TUSD has a multiplicative return of $0.9995 \times 1.0042 \approx 1.0037$. Starting with \$10,000, the final capital is approximately \$10,037, yielding a profit of \$37. The total log-space cost is $-\log(0.9995) + (-\log(1.0042)) \approx 0.0005 - 0.0042 = -0.0037$, which is negative, indicating a profitable path. This example illustrates both inter-exchange transfer and intra-exchange trade edges, and shows that profitable paths need not form closed cycles.

4. Novel Heuristics for Arbitrage Search

We design three guidance heuristics (h_1 , h_2 , h_4) and one search-control strategy (h_3) to steer A* toward executable arbitrage routes. Each captures a different dimension of execution feasibility via an additive penalty $h(n) \geq 0$ in $f(n) = g(n) + h(n)$. As discussed in Section 3, these are *guidance penalties*, not admissible lower bounds. Full hyperparameter values are reported in Appendix A.1.

4.1. h_1 – Liquidity Heuristic

The liquidity heuristic penalizes low-volume markets that cannot support the required trade size. We compute a liquidity ratio from 24-hour quote volume, the remaining arbitrage window T_{remain} , and order size:

$$\text{liquidity_ratio} = \frac{(24h_vol/86400) \cdot T_{\text{remain}}}{\text{order_size_usd}}, \quad h_1(n) = \lambda_{liq} \left(1 - \text{clamp} \left(\frac{\log_{10}(\text{liquidity_ratio}) + 1}{2}, 0, 1 \right) \right).$$

This heuristic favours deep, actively traded markets and is time-aware.

Table 1. Baseline algorithm performance on cached graph (41 nodes, 864 edges, 30 start nodes). All profits are net after trading, withdrawal, and gas fees.

Baseline	Cash	Succ.	Profit (\$)	Profit (%)	Expand	Time (s)	Hops
Dijkstra ($h=0$)	\$1k	56.7%	1.00	0.100	359	0.010	3
	\$10k	56.7%	10.01	0.100	359	0.010	3
	\$100k	56.7%	100.07	0.100	359	0.010	3
3-hop enum.	\$1k	30.0%	0.54	0.054	346	<0.001	3
	\$10k	30.0%	5.40	0.054	346	<0.001	3
	\$100k	30.0%	54.04	0.054	346	<0.001	3
simple_1hop	All	0/30	–	–	–	<0.001	1
simple_2hop	All	0/30	–	–	–	<0.001	2
Bellman–Ford	All	0/30	–	–	–	<0.001	–

5.1. Data Sources and Graph Construction

We gather live market data from twelve major exchanges (Binance, Kraken, KuCoin, Bybit, OKX, Gate.io, Bitget, MEXC, HTX, Coinbase, Crypto.com, Phemex) via CCXT [1]. Prices are normalized to USD using conservative bid/ask estimates, and a $\pm 2\%$ price tolerance filter excludes depegged assets. The fee model captures taker fees, exchange-specific withdrawal fees per blockchain, and estimated gas fees. Transfer time is modelled as blockchain confirmation time plus 45s user execution overhead. All hyperparameters are listed in Appendix A.1.

5.2. Ablation Studies

We isolate heuristic contributions via three ablation baselines: (1) **Cycle ablation** using Bellman–Ford negative-cycle detection; (2) **Depth ablation** with `simple_1hop` (single action) and `simple_2hop` (minimal two-edge path); (3) **Heuristic ablation** setting $h(n) = 0$ (Dijkstra). Bellman–Ford detects whether negative-weight cycles exist in the log-transformed graph but does not account for slippage, latency, or reliability, and fails to produce executable paths under realistic frictions. The shallow baselines fail across all start nodes, confirming that profitable stablecoin arbitrage requires ≥ 3 hops. Results are in Table 1.

5.2.1. Cached-Graph Heuristic Comparison

We compare all heuristics on a cached graph (41 nodes, 864 edges, 30 start nodes, three cash levels), isolating search from API I/O. Table 2 reports aggregate performance.

h_2 achieves a consistent **29% reduction in node expansions** relative to Dijkstra while matching profit within 1% at \$100k. h_4 expands 87% more nodes but at comparable wall-clock time (0.020s vs. 0.010s) due to zero API overhead, producing longer risk-aware paths. h_1 behaviour is cash-dependent: 11% fewer expansions at \$1k but 99% more at \$100k.

5.3. Evaluation Methodology

We evaluate along five dimensions: cached-graph heuristic comparison, graph scaling, temporal robustness (8-hour overnight campaign), quote staleness under execution delay, and hyperparameter sensitivity. *Cached* experiments freeze the graph from a single snapshot, isolating algorithmic differences; *live* experiments include API I/O overhead. We report success rate, conditional average profit, node expansions, wall-clock time, path length, and path type (intra- vs. cross-exchange).

Table 2. Cached-graph heuristic comparison (41 nodes, 864 edges, 30 start nodes per cash level). Expansion reduction is relative to Dijkstra.

Heuristic	Cash	Success	Avg Profit (\$)	Avg Expand	Avg Time (s)	Exp. Δ vs Dijkstra
Dijkstra ($h=0$)	\$1k	56.7%	1.00	359	0.010	—
	\$10k	56.7%	10.01	359	0.010	—
	\$100k	56.7%	100.07	359	0.010	—
h_1 (liquidity)	\$1k	56.7%	0.67	319	0.172	+11.2%
	\$10k	56.7%	6.72	568	0.016	−58.0%
	\$100k	56.7%	63.11	715	0.021	−98.9%
h_2 (slippage)	\$1k	56.7%	0.99	256	0.221	+28.8%
	\$10k	56.7%	9.91	256	0.011	+28.8%
	\$100k	56.7%	100.07	252	0.012	+29.9%
h_4 (Weighted A*)	\$1k	56.7%	0.71	673	0.020	−87.4%
	\$10k	56.7%	7.06	673	0.022	−87.4%
	\$100k	56.7%	70.58	673	0.022	−87.4%
3-hop enum.	\$1k	30.0%	0.54	346	0.000	+3.6%
	\$10k	30.0%	5.40	346	0.000	+3.6%
	\$100k	30.0%	54.04	346	0.000	+3.6%

Table 3. Mean node expansions by graph size and heuristic (10 trials each).

Exch.	$ V $	$ E $	Dijkstra	h_1	h_4	3-hop
4	19	161	341	407	600	365
6	28	368	378	758	1100	662
8	35	619	387	479	618	378
10	39	760	404	577	706	490
12	41	864	395	768	910	67

5.4. Results

5.4.1. Deterministic Heuristic Comparison

On the cached graph (41 nodes, 864 edges, 30 start nodes, three cash levels), all A*-based methods achieve identical 56.7% success rates, confirming that heuristic choice affects *which* path is found, not *whether* one exists (Table 2). The 3-hop baseline succeeds for only 30% of start nodes. Start-node sensitivity persists: $\sim 43\%$ of nodes yield no profitable path regardless of method.

5.4.2. Comparison Against Baselines

The 3-hop enumeration baseline finds the same optimal paths as A* when 3-hop paths exist, confirming no path-quality sacrifice. Its complexity $O(|V| \cdot d^3)$ becomes prohibitive for larger graphs (Section 5.4.3). Simple 1-hop and 2-hop baselines and Bellman–Ford all fail to produce profitable paths, validating that our execution-aware graph eliminates theoretical-only opportunities.

5.4.3. Graph Scaling

We scale from 4 to 12 exchanges (19–41 nodes, 161–864 edges, 10 trials each). Table 3 shows that Dijkstra’s expansions remain flat (341–404) while h_1 and h_4 expand 19–130% more nodes due to penalty-driven exploration. The 3-hop baseline exhibits high variance (67–662 evaluations), contrasting with monotonic A* scaling.

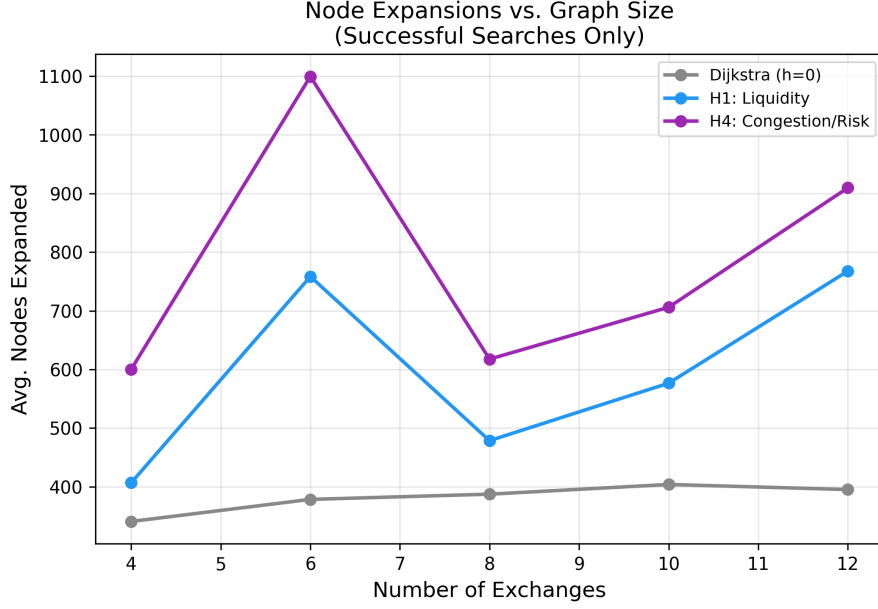


Figure 3. Mean node expansions vs. number of exchanges. Heuristic-guided methods expand more nodes than Dijkstra due to penalty-driven exploration.

Table 4. Quote staleness: path survival and profit change. Negative decay indicates profits increased.

Delay (s)	Paths	Profitable	Mean Decay (%)
5	45	100%	−64.6
10	45	100%	−67.0
30	45	100%	−71.6
60	45	100%	−72.7
120	45	100%	−72.4
300	45	98%	−55.0

5.4.4. Quote Staleness and Path Survival

We re-evaluate discovered paths against fresh market data after delays of 5–300 s across 270 measurements. **99.6%** of paths remain profitable up to 120 s; only at 300 s does a single path lose profitability (98% survival). This demonstrates strong *structural* persistence of stablecoin arbitrage paths, driven by the slowly-changing fee landscape. We note that CCXT-level ticker caching may mask true price-change risk; historical tick-data replay would strengthen this finding (see Table 4 and Figure 4).

5.4.5. Temporal Robustness: 8-Hour Overnight Test

We run all five methods from 5 start nodes at three cash levels over 96 snapshots (8 hours, 5-min intervals), yielding **7,200 search instances**. All A*-based methods achieve **100% success**; 3-hop enumeration succeeds in only 39.8% of runs. Table 5 shows that h_2 produces the highest cross-exchange path ratio (95.7%), while h_4 and h_1 more frequently settle on intra-exchange trades.

Restricting to cross-exchange paths, Dijkstra achieves the highest mean profit (\$42.42), followed by h_2 (\$39.15), h_1 (\$34.97), h_4 (\$22.16), and 3-hop (\$20.76). The key differentiator

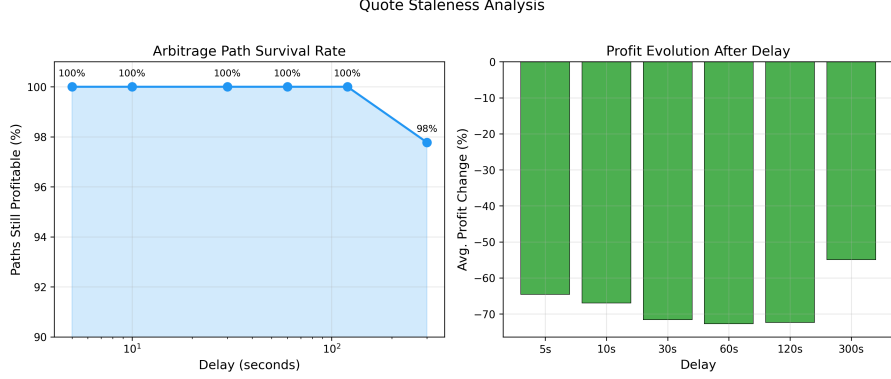


Figure 4. Quote staleness: profit change and survival rate vs. re-evaluation delay (5–300 s).

Table 5. Overnight path-type breakdown (1,440 runs each).

Method	Cross-Exch.	Intra-Exch.	Cross %
h_2 (slippage)	1,378	62	95.7%
Dijkstra	1,254	186	87.1%
h_1 (liquidity)	966	474	67.1%
h_4 (Weighted)	849	591	58.9%
3-hop enum.	573	0	100%

is *latency*: h_4 achieves P99 = 33 ms (comparable to Dijkstra’s 17 ms) via pre-computed scores, while h_1 and h_2 suffer P99 latencies of 6.8–6.9 s from live API calls, making h_4 the only viable heuristic for real-time deployment. h_2 achieves the fewest mean expansions (269 vs. Dijkstra’s 422). Arbitrage opportunities persist across all 96 snapshots, with profits clustering around 2–3 structural optima per start node (Figure 5).

5.4.6. Sensitivity Analysis

We conduct parameter sweeps on the 12-exchange graph. h_1 penalty weight. Sweeping $\lambda_{\text{liq}} \in \{0.1, 0.5, 1.0, 2.0, 5.0\}$ (25 trials each), profit and success rate are invariant (80%, \$27.77). Expansions vary by only 5.6%, indicating the penalty primarily reorders the open list.

h_2 slippage parameters. A joint sweep of $(\lambda_{\text{slip}}, \theta)$ across 100 configurations shows a broad plateau of high performance at moderate values ($\lambda_{\text{slip}} \in [0.3, 1.0]$, $\theta \in [5, 20]$ bps), confirming h_2 is not sensitive to precise tuning (Figure 6).

Order size and path depth. Profit scales linearly from \$0.09 at \$100 to \$86.18 at \$100k, with constant 80% success rate (Figure 7). Varying depth from 3 to 6 hops yields identical optimal paths, confirming profitable paths are inherently short (3 hops).

5.5. Ethics and Broader Impact

This work uses only publicly available market data; no proprietary or user-specific data are used. Our framework is a research prototype not deployed for live trading. A detailed discussion of market impact, execution risk, environmental considerations, and regulatory compliance is provided in Appendix A.2.

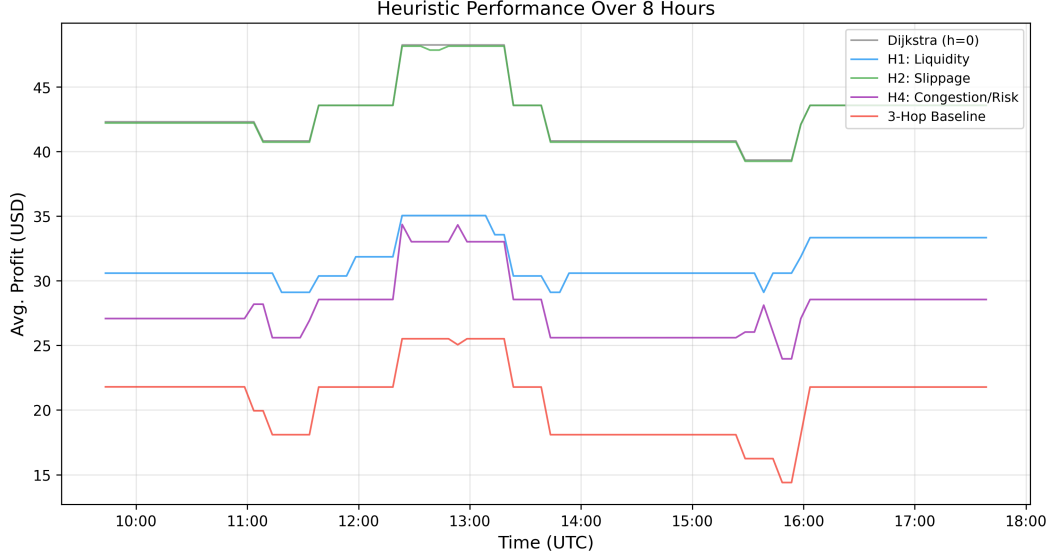


Figure 5. Overnight heuristic comparison: mean profit, success rate, and node expansions across 1,440 runs per method.

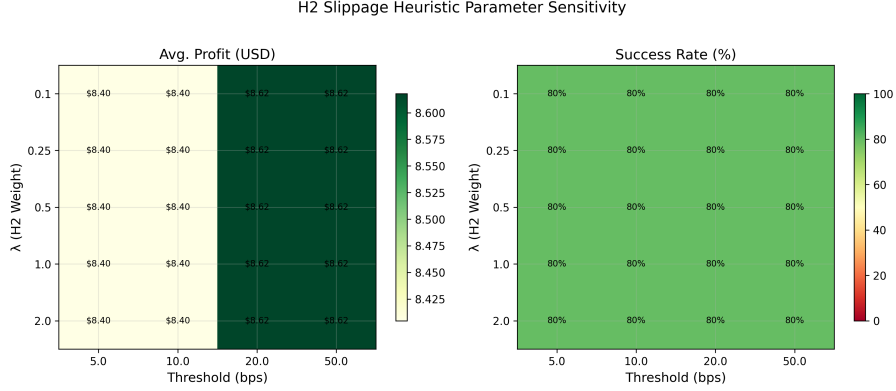


Figure 6. h_2 sensitivity: success rate across $(\lambda_{\text{slip}}, \theta)$ grid. A broad plateau of high success exists around moderate parameter values.

6. Conclusions

We presented an execution-aware A* search framework for cross-exchange stablecoin arbitrage that integrates real-world trading costs (fees, slippage, withdrawal costs, gas, transfer latency, and exchange reliability) directly into graph-based pathfinding. Three guidance heuristics (h_1 , h_2 , h_4) and a multi-start strategy (h_3) were evaluated across **7,200 overnight instances**, sensitivity sweeps, and graph-scaling trials on 12 exchanges (41 nodes, 864 edges).

Core contribution. The execution-aware graph formulation is the primary contribution: by encoding all frictions into edge weights via $w(e) = -\log(r(e))$, even Dijkstra ($h=0$) achieves 100% overnight success with \$42.87 mean profit, establishing that the modelling, not any single heuristic, drives value.

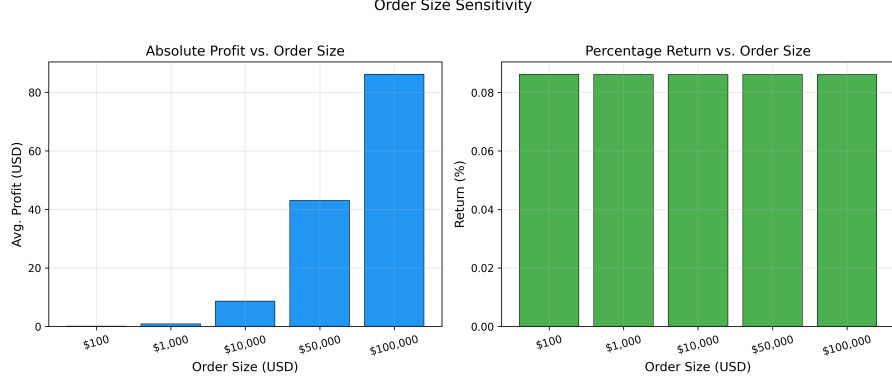


Figure 7. Mean profit vs. order size (\$100–\$100k). Profit scales linearly; success rate is constant at 80%.

Heuristic roles. h_2 (slippage) is most effective for search efficiency (29–36% expansion reduction, 95.7% cross-exchange paths) but suffers $P99 = 6.8s$ latency from live API calls. h_4 (Weighted A*) is best for real-time deployment ($P99 = 33ms$, zero API overhead). h_1 (liquidity) is useful at small order sizes but counterproductive at large ones. Empirical findings. Profitable paths are short (2–3 hops) with $\sim 0.09\%$ net returns. Opportunities persist across all 96 overnight snapshots. Sensitivity analysis confirms robustness to hyperparameters and linear profit scaling with order size.

6.1. Future Work and Limitations

Key directions include: (1) asynchronous order-book pre-fetching to combine h_2 ’s pruning quality with h_4 ’s latency; (2) extension to DEXs/AMMs with deterministic bonding-curve slippage; (3) historical tick-data replay for rigorous staleness analysis; (4) adaptive heuristic selection based on data freshness.

The framework has not executed live trades; planned paths may change before execution. Approximately 20% of discovered paths are intra-exchange trades discoverable by trivial price comparison. Exchange reliability scores are static priors that may not capture sudden failures.

Acknowledgments

References

- [1] CCXT. *CryptoCurrency eXchange Trading Library*. Accessed: 2024. 2024. URL: <https://github.com/ccxt/ccxt>.
- [2] R. Oantă and A. Coroiu. “Crypto Advisor: A Web Application for Spotting Cross-Exchange Cryptocurrency Arbitrage Opportunities”. In: *Proceedings of the 15th International Conference on Computer Supported Education (CSEDU 2023)*. Vol. 1. 2023, pp. 238–246. ISBN: 978-989-758-641-5. DOI: [10.5220/0011850400003470](https://doi.org/10.5220/0011850400003470).
- [3] R. Bellman. “On a routing problem”. In: *Quarterly of Applied Mathematics* 16.1 (1958), pp. 87–90.
- [4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. 3rd. MIT Press, 2009.
- [5] L. Zhou, K. Qin, C. F. Torres, D. V. Le, and A. Gervais. “On the Just-In-Time Discovery of Profit-Generating Transactions in DeFi”. In: *IEEE Symposium on Security and Privacy (Oakland)*. 2021.

- [6] Anonymous. “An Improved Algorithm to Identify More Arbitrage Opportunities”. In: *arXiv preprint arXiv:2406.16573* (2024).
- [7] Anonymous. “Real-Time Identification of Negative Cycles for High-Efficiency Arbitrage”. In: *VLDB 18* (2025), pp. 4081–4094.
- [8] F. Fang, C. Ventre, M. Basios, L. Kanthan, D. Martinez-Rego, F. Wu, and L. Li. “Cryptocurrency trading: a comprehensive survey”. In: *Financial Innovation* 8.1 (2022), pp. 1–59.
- [9] M. Kearns and Y. Nevmyvaka. “Reinforcement Learning for Optimized Trade Execution”. In: *International Conference on Machine Learning (ICML)*. 2006.
- [10] Anonymous. “Optimal Execution with Reinforcement Learning”. In: *arXiv preprint arXiv:2411.06389* (2024).
- [11] A. Obadia, A. Salles, L. Sanchez, T. Sri, V. Buterin, and P. Daian. “Unity is Strength: A Formalization of Cross-Domain Maximal Extractable Value”. In: *arXiv preprint arXiv:2112.01472* (2021).
- [12] E. Nikolova and D. R. Karger. “Route Planning under Uncertainty: The Canadian Traveller Problem Revisited”. In: *Proceedings of the 23rd National Conference on Artificial Intelligence (AAAI)*. 2008, pp. 969–974.
- [13] S.-H. Lim, I.-J. Rhee, and M. C. Fu. “Risk-averse shortest path problems”. In: *Mathematics of Operations Research* 47.3 (2022), pp. 2001–2029.
- [14] R. Almgren and N. Chriss. “Optimal execution of portfolio transactions”. In: *Journal of Risk* 3.2 (2001), pp. 5–39.
- [15] I. Pohl. “Heuristic search viewed as path finding in a graph”. In: *Artificial Intelligence* 1.3-4 (1970), pp. 193–204.

Appendix A. Appendix

A.1. Hyperparameter Configuration

Table 6 lists all hyperparameters used in the heuristic functions and search algorithms. Values are fixed across all experiments unless otherwise noted.

Table 6. Hyperparameters and their values.

Parameter	Description	Value
<i>H1 – Liquidity Heuristic</i>		
λ_{liq}	Liquidity penalty weight	1.0
Unknown penalty	No-data fallback cost	5.0
Cache TTL	Volume cache lifetime	60 s
<i>H2 – Slippage Heuristic</i>		
λ_{slip}	Slippage penalty weight	0.5
θ	Slippage tolerance	10 bps
Unknown penalty	No-data fallback cost	50.0
Cache TTL	Order-book cache lifetime	30 s
<i>H4 – Chain/Exchange Risk</i>		
λ_{chain}	Chain risk penalty weight	1.0
$\lambda_{\text{exchange}}$	Exchange risk penalty weight	1.0
w_{min}	Min Weighted A* weight	1.0
w_{max}	Max Weighted A* weight	3.0
<i>Search Parameters</i>		
Max depth	Maximum path hops	6
T_{max}	Arbitrage time window	1800 s
Early exit iter.	Iters after first profit	200
k (H3)	Multi-start parallel runs	3
User overhead	Execution latency per hop	45 s

Chain transfer times. Transfer-time estimates are drawn from documented blockchain finality times plus typical exchange processing delays: Solana ≈ 1 s, BNB Chain ≈ 4 s, Polygon ≈ 5 s, Tron ≈ 30 s, Arbitrum/Base ≈ 120 s, Ethereum L1 ≈ 600 s. These are used by h_4 to compute chain reliability penalties and by the transfer-time model to assess path time-feasibility.

Exchange reliability scores. Each exchange is assigned a static reliability score $s(n) \in [0, 1]$ based on historical uptime, withdrawal availability, and community reports. Higher scores indicate more reliable exchanges. These scores are pre-computed and cached, requiring zero API calls during search.

A.2. Ethics and Broader Impact

Data and privacy. This work uses only publicly available market data (prices, order books, fee schedules) accessed through CCXT. No proprietary, personal, or user-specific data are collected or used.

Market impact. Arbitrage activity generally improves market efficiency by reducing cross-exchange price discrepancies. However, large-scale automated arbitrage could theoretically contribute to flash crashes or liquidity draining in thin markets. Our framework operates at order sizes (\$100–\$100k) that are small relative to exchange daily volumes (\$100M+) and is designed as a research prototype, not a production trading system.

Environmental considerations. Our framework queries exchange APIs and performs light-weight graph search computations. It does not execute on-chain transactions, mine blocks, or contribute to blockchain energy consumption. The computational footprint is negligible (P99 search time < 7 s, even for API-dependent heuristics).

Regulatory compliance. Cross-exchange arbitrage operates within established market structures and does not exploit insider information or engage in market manipulation. Users of this framework should comply with applicable financial regulations in their jurisdiction, including exchange terms of service and cryptocurrency trading laws.

A.3. Additional Experimental Results

Heuristic comparison: expansions and compute time. Figure 8 compares mean node expansions and compute time across all heuristics on the cached graph. h_2 consistently achieves the fewest expansions, while h_4 matches Dijkstra-class latency due to pre-computed scores requiring zero API calls.

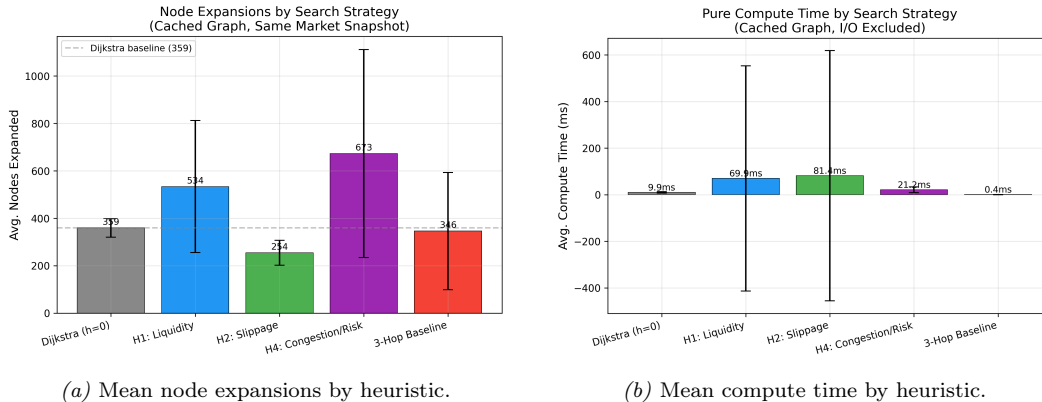


Figure 8. Heuristic comparison on cached graph (30 start nodes, 3 cash levels).

Overnight temporal evolution. Figure 9 shows profit trajectories across 96 snapshots (8 hours, 5-min intervals). Profitable opportunities persist throughout the entire session, with profits clustering around 2–3 structural optima per start node. Periodic dips correspond to temporary market equilibria that resolve within minutes.

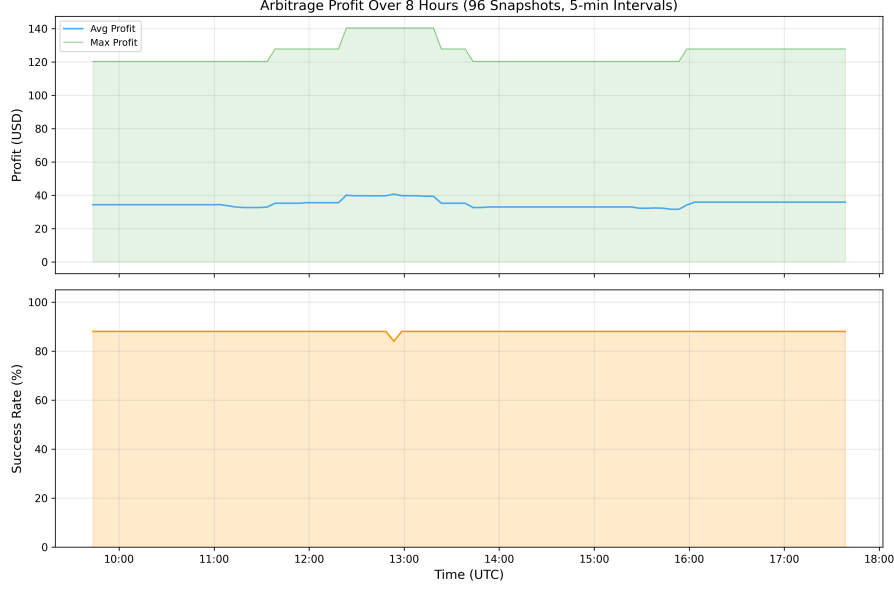


Figure 9. Overnight profit timeseries across all heuristics (8 hours, 5-min intervals, 5 start nodes, 3 cash levels).

Sensitivity: h_1 liquidity penalty and max depth. Figure 10 shows that h_1 performance is robust across $\lambda_{liq} \in [0.1, 5.0]$, and that increasing max depth beyond 3 hops yields no additional profit, confirming that profitable stablecoin paths are inherently short.

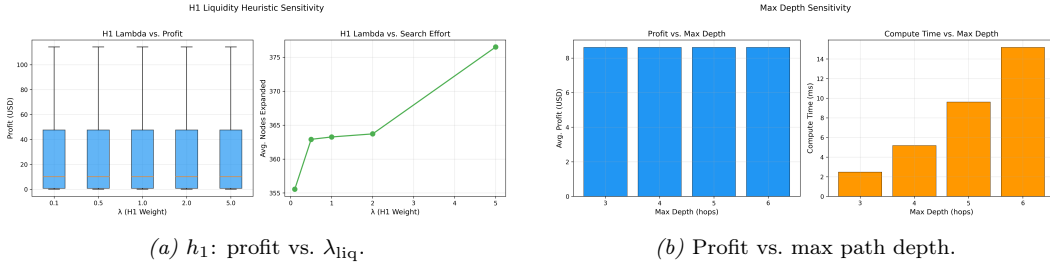
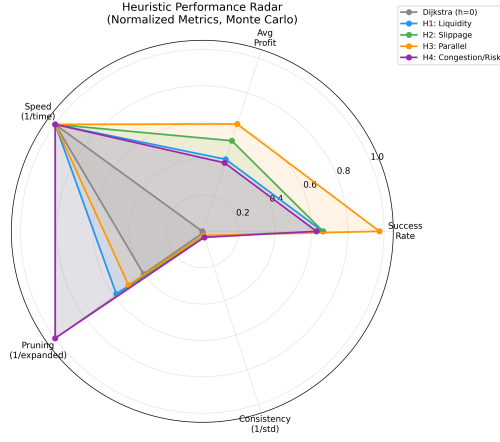


Figure 10. Additional sensitivity analysis.

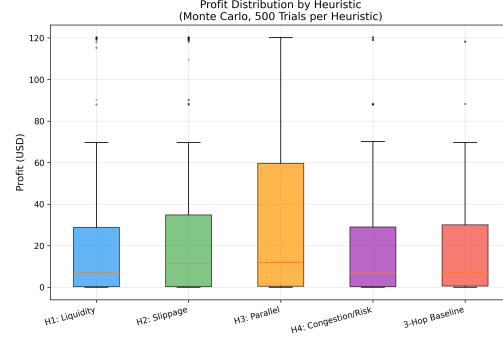
Radar summary and profit distribution. Figure 11 provides a multi-dimensional summary of heuristic performance across six metrics, alongside the profit distribution across order sizes. h_4 dominates on speed and reliability, while h_2 leads on expansion efficiency.

A.4. Interactive Visualization System

We provide a Streamlit-based web interface for real-time exploration of arbitrage opportunities. The UI integrates live CCXT market data and provides four tabs: (1) **Arbitrage Graph** with interactive network visualization, heuristic selection, and search execution with



(a) Radar: multi-metric heuristic comparison.



(b) Profit distribution by heuristic.

Figure 11. Overall heuristic performance summary.

streaming log output; (2) **Live Prices** showing real-time stablecoin prices across exchanges; (3) **Fees** displaying all fee structures; (4) **Documentation**. Upon search completion, results include the complete route, per-node heuristic values, and step-by-step execution details with fees and transfer times. The system uses NetworkX for graph operations and Matplotlib for visualization. Source code is available in the anonymized repository.